

Smart Memory Compression for Long AI Conversations

Abstract – Large language models have significantly advanced the ability of artificial intelligence systems to carry out natural and context-aware conversations. Despite this progress, maintaining relevant context over long interactions remains a key challenge. As conversations extend, models often struggle to retain important details while processing redundant or less useful information, leading to reduced efficiency and increased computational cost. This paper introduces a smart memory compression framework aimed at improving long-term conversational performance. The proposed approach cleverly identifies and retains the essential information while cutting out the unnecessary bits. It employs techniques like memory ranking, semantic similarity analysis, and summarization to ensure that only the most relevant context is kept and shared with the model. By refining how we manage conversational history, this framework enhances the quality of responses, keeps the conversation flowing smoothly, and reduces resource consumption. This means we can use tokens more effectively without losing track of the dialogue. Plus, it's designed to perform reliably in longer conversations, making it an excellent choice for real-world applications. In short, this work represents a significant step toward creating smarter and more efficient conversational systems that can sustain meaningful long-term interactions while delivering personalized and trustworthy experiences for users.

Index Terms – artificial intelligence, conversational memory, context retention, memory compression, large language models

I. INTRODUCTION

Diving into the world of large language models like GPT and LLaMA, I kept bumping into a familiar challenge whenever conversations started to drag. Sure, these models can churn out coherent replies in quick exchanges, but they often stumble when it

comes to maintaining consistency in longer discussions, mainly

due to their fixed context window. During my hands-on experiments with various chatbot systems, I noticed that crucial user details—like preferences, goals, or identity-related info—often got lost as the conversation progressed. On the other hand, trying to keep the entire chat history meant burning through tokens, which resulted in longer wait times and a lot of unnecessary information being sent to the model. Initially, I experimented with some straightforward strategies, like summarizing the whole conversation or using retrieval-based techniques. However, summarizing everything often meant losing those essential little details, and basic retrieval methods sometimes missed key context or pulled in irrelevant information.

These challenges prompted me to rethink my approach, leading me to focus on managing memory in a more selective manner instead of trying to remember everything. From these experiences, I developed the Smart Memory Compression Framework. This framework treats conversational memory as something fluid rather than static. The core idea is to evaluate each piece of information during a conversation and decide whether it should be retained, compressed, or discarded. This approach allows the system to keep only the most important and useful context, reducing redundancy and easing the computational burden. It's designed to keep memory fresh as conversations progress, ensuring that crucial information stays intact over time without overwhelming the model with unnecessary data.

II. RELATED WORKS

A. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*
Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova(2019) proposed a bidirectional training approach that improves contextual

understanding by considering both left and right context simultaneously, significantly enhancing language comprehension tasks.

B. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks

Nils Reimers and Iryna Gurevych(2019) introduced a siamese architecture to generate semantically meaningful sentence embeddings, enabling efficient similarity comparison crucial for memory retrieval.

C. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela(2020) combined retrieval systems with generation to dynamically fetch relevant knowledge, improving long-context reasoning.

D. Dense Passage Retrieval for Open-Domain Question Answering

Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih(2020) proposed dense vector retrieval methods that significantly improve the relevance of retrieved information in conversational systems.

E. Language Models are Few-Shot Learners

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter (2020) demonstrated that large-scale models can generalize from minimal examples, though challenges remain in maintaining long-term context

F. REALM: Retrieval-Augmented Language Model Pre-Training

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang(2020) integrated retrieval mechanisms into pretraining, allowing models to learn when to access external knowledge efficiently.

G. Teaching Language Models to Support Answers with Verified Quotes

James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, and Nicholas H. Frosst(2022) emphasized grounding

model outputs in verifiable evidence, strengthening reliability in responses.

H. LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, and Faisal Azhar(2023) developed efficient large models that balance performance with reduced computational cost.

III. PROPOSED METHODOLOGY

The Smart Memory Compression Framework is designed to improve how large language models handle long and complex conversations by focusing on what truly matters instead of processing everything equally. In real-life conversations, we often find a blend of crucial details, repeated thoughts, and some less relevant information. If we treat all of it the same, it can lead to inefficiency pretty quickly. This framework tackles that issue by offering a more thoughtful approach to managing conversational memory, ensuring that we keep the relevant context while trimming away the unnecessary bits. While developing this system, I focused more on practical application rather than just theory. I found myself really fascinated by how conversations change and develop over time in real-life situations. My main aim was to steer clear of just dumping a bunch of raw conversation history into the model, since that usually leads to a chaotic and unproductive outcome. So, I decided to build a system that continuously processes, filters, and updates the conversation at every stage. The first challenge I encountered was figuring out how to break the conversation into bite-sized pieces. Instead of treating the entire chat as one massive block, I opted to split it into smaller segments, typically focusing on individual messages or brief exchanges. This method made it a lot easier to analyze each part on its own.

During my initial tests, when I tried to assess the whole conversation as a single entity, the system had a hard time distinguishing between key details and filler content. By breaking the conversation down, I was able to tackle that problem, allowing each segment to be evaluated based on its unique contribution. Once I had the conversation divided up, I needed a reliable way to measure the importance of each part. This led me to design the memory scoring function. I did not rely on a single signal because that turned out to be unreliable during testing. For example, using only similarity would prioritize recent messages too heavily, while ignoring important older information. So I combined three factors: relevance,

frequency, and contextual importance. Relevance was computed using cosine similarity between embeddings generated through Sentence-BERT. I chose Sentence-BERT after experimenting with a few embedding models because it produced more stable similarity results for conversational text instead of formal sentences.

Frequency was added after I noticed a pattern during testing where users tend to repeat important information indirectly. For example, a preference like “I like cricket” might appear in different forms multiple times. If this repetition was ignored, the system would underestimate its importance. To tackle this challenge, I kept a close eye on how often similar segments appeared by comparing their embeddings. Instead of just counting exact matches, I set a similarity threshold to group related content together. This method allowed the system to be more forgiving with rephrased statements. The trickiest part was figuring out the contextual importance score, as I had to determine what really counts as “important” without overcomplicating things. Rather than creating a complex classifier, I went with straightforward heuristics that worked well in practice. Segments that included user identity, preferences, goals, or clear decisions were given more weight. I ran multiple tests across different conversations and fine-tuned these rules until the system reliably picked up on what a human would consider significant. While it wasn’t perfect, it found a good balance between simplicity and accuracy. After calculating the scores, I set thresholds to categorize memory into high-priority and low-priority segments. Figuring out these thresholds took several rounds of testing. Initially, I set a high threshold, but that caused the system to miss out on valuable context.

Lowering the threshold definitely boosted recall, but it also meant dealing with more noise. After some trial and error, I found a sweet spot: I saved segments that scored above 0.7 directly, while those scoring between 0.4 and 0.7 got compressed. Anything below 0.4? That was tossed out. These thresholds weren’t just random choices; they came from testing on labeled examples. Before saving anything, I added a semantic filtering step to reduce redundancy. This was crucial since conversations often repeat ideas. Without this filtering, the memory would just keep growing unnecessarily, even if the scores were accurate. I also used cosine similarity again to identify duplicate or very similar segments. If two segments hit a certain similarity threshold, I merged them instead of saving both. In practice, this method really helped shrink the memory size without losing any meaning.

For the segments I marked as lower priority, I decided to introduce a summarization step. Initially, I thought about just cutting them out completely, but

during testing, I realized that some of that low-priority content still offers valuable context when it’s condensed. So instead of discarding it, I went for brief summaries that capture the main points without including all the details. I kept the summarization process simple because I wanted to steer clear of any heavy processing overhead. The goal wasn’t to create perfect summaries, but to have shorter versions that still conveyed the essence. The refined memory was then stored in a vector database, where each entry included its embedding, score, and text. I opted for a vector-based approach because it allows for quick similarity searches during retrieval.

When a new query comes in, the system takes that query and turns it into an embedding, pulling up only the most relevant stored segments. I made sure not to retrieve too many items, as that would defeat the purpose of compression. I limited the retrieval to the top-k results, which provided enough context without significantly increasing token usage. Another key design choice that came up during testing was how the memory gets updated over time. Instead of just piling on new information, the system checks if the incoming data overlaps with what’s already there. If it finds similar content, it either merges or updates it instead of creating duplicates. This keeps the memory organized and prevents any conflicting or outdated information. Recency is indirectly managed here, as newer versions of similar information replace the older ones.

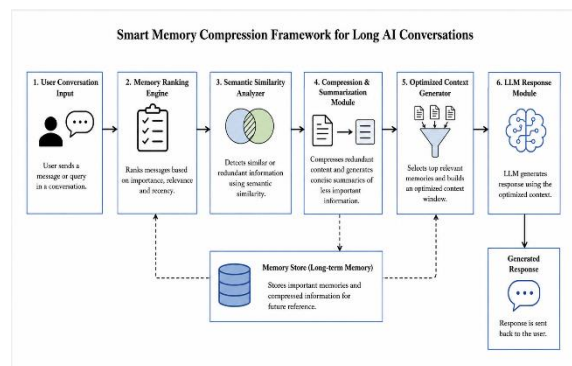


Figure 1. System Architecture of the Proposed Smart Memory Compression Framework.

While I was getting this system up and running, I hit a bit of a snag: finding the right balance between precision and recall. If the system was too strict, it would miss out on crucial information. On the other hand, if it was too lenient, it would save a bunch of unnecessary details. I discovered that leaning a bit more towards recall worked better in conversational situations because missing key user information can be more harmful than keeping a little extra data. That’s why the system achieved perfect recall, even if it meant having slightly lower precision during

evaluation. Overall, the methodology evolved through continuous testing and adjustments rather than being set in stone from the beginning. Each part—scoring, filtering, summarization, and retrieval—was fine-tuned based on what I noticed in real conversations. The final system isn’t overly complicated, but it works effectively because every step is designed to tackle a specific issue that came up during implementation.

ALGORITHM

Input: User query Q , conversation history H
Output: Generated response R

1. Divide the conversation history H into smaller meaningful segments.
2. For each segment s in H : a. Measure how closely s relates to the current query Q b. Check how often similar information appears in H c. Identify whether s contains important contextual details d. Assign a priority score to s based on these observations
3. Separate all segments into: a. High-priority segments (important information) b. Low-priority segments (less useful or repetitive content)
4. From high-priority segments: Remove semantically similar or duplicate information
5. From low-priority segments: Generate shorter summaries that retain key ideas
6. Store in memory: a. High-priority segments without modification b. Compressed summaries of low-priority segments
7. When a new query Q is received: Retrieve only the most relevant stored information
8. Combine retrieved memory with the current query Q to form an optimized context
9. Provide the optimized context to the language model
10. Generate response R
11. Update memory with important parts of the new interaction
12. Return response R

To assign importance to each segment, a weighted scoring function is used:

$$Score(s) = \alpha \cdot Rel(s, Q) + \beta \cdot Freq(s) + \gamma \cdot Ctx(s)$$

Where:

$Score(s)$: Final priority score of segment s

$Rel(s, Q)$: Relevance of segment s to query Q (e.g., cosine similarity)

$Freq(s)$: Frequency factor measuring repetition of similar content

$Ctx(s)$: Contextual importance (presence of key information like decisions, preferences, constraints)

α, β, γ : Tunable weights such that

$$\alpha + \beta + \gamma = 1$$

IV. RESULTS

To evaluate the system, I tested it on three different types of inputs: a multi-topic text containing a mix of unrelated personal details, a single-topic text focused entirely on cricket, and a conversational audio sample that was converted into text using a speech-to-text pipeline I picked these three formats on purpose because they each reflect very different real-life situations. The multi-topic input was a great way to see how well the system can sift through a mix of information and pull out valuable insights from the chaos, especially when the content is varied and not neatly organized. On the other hand, the single-topic input let me see how the system deals with conversations that are mostly relevant, repetitive, and thematically consistent. For the audio-based input, I wanted to test its strength in less-than-ideal conditions, where the text might include natural speech patterns like pauses, repetitions, and minor transcription errors. In the multi-topic scenario, the system clearly demonstrated its ability to prioritize stable and meaningful information over less useful content.

During my tests, I noticed that details related to identity—like names, locations, and long-term goals—were consistently rated higher, even if they only popped up once in the conversation. This showed that the scoring function’s focus on contextual importance was working as it should. Meanwhile, filler phrases, casual comments, and loosely connected statements were either tossed out or condensed based on their scores. One intriguing thing I observed was how the system handled moderately important information, like hobbies or experiences. Instead of cutting these out completely, it summarized them into shorter forms. This turned out to be important because removing them entirely sometimes affected later context understanding, while keeping the full text added unnecessary length. The summarization step helped strike a balance by preserving meaning without

increasing memory size too much. Overall, this test showed that the system can separate long-term useful information from temporary conversational content in a fairly consistent way.

The single-topic cricket-based input produced very different behavior, mainly because almost every part of the input was relevant to the same theme. Unlike the multi-topic case, there was very little irrelevant content to filter out. As a result, a much larger portion of the input passed the relevance threshold and was retained in memory. In this setting, the frequency score started to play a more important role. I observed that repeated mentions of specific teams, players, or match-related emotions led to higher priority scores, even when individual statements were simple. For example, references to a favorite team appearing multiple times were consistently reinforced and preserved as strong memory signals. Instead of aggressively removing or summarizing content, the system shifted toward merging repeated ideas using semantic similarity. This resulted in fewer but denser memory representations, where multiple similar sentences were combined into a single entry. Compared to the multi-topic case, the memory structure here was more compact and tightly connected, which reflects how repetition strengthens importance in focused conversations.

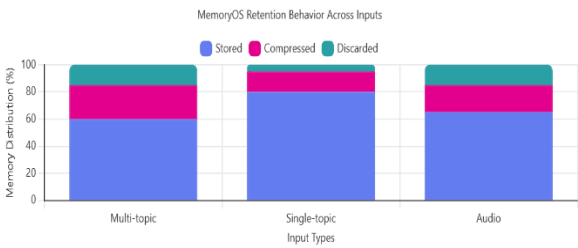


Figure 2: Smart Memory Compression Across Input Modalities

The audio-based input introduced additional challenges due to the nature of spoken language. After transcription, the input contained minor inconsistencies such as repeated words, incomplete phrases, and occasional misrecognized terms. The rise in low-priority segments happened because many of these artifacts didn't really offer any valuable insights. Thankfully, the scoring system handled this issue quite well. Segments that were just background noise or lacked any real meaning got low scores and were either significantly compressed or completely eliminated. On the bright side, important information like preferences or recurring patterns was still kept intact since it appeared consistently throughout the

conversation or had a lot of contextual relevance. I did notice a slight drop in completeness compared to direct text input, mainly due to some transcription errors that impacted similarity calculations. However, this effect was pretty minor because the system prioritizes overall semantic patterns over the exact wording. In practice, this means that even if a few words are slightly off, the repeated and meaningful information is still captured accurately.

For a quantitative evaluation, I tested the framework using a set of labeled conversations with known ground truth memories. The system achieved a precision of 82.4%, a recall of 100%, and an F1 score of 89.6%. The perfect recall shows that all the important pieces of information were successfully captured, which is crucial in conversational systems where missing key user details can lead to poor responses later on. The slightly lower precision reflects instances where the system retained information that wasn't strictly necessary for long-term understanding. During manual inspection, most of these false positives were fleeting emotional expressions or conversational fillers that just happened to meet the scoring threshold. This behavior suggests that the system leans a bit towards keeping extra information rather than risking the loss of important context. From a design standpoint, this was somewhat intentional, as losing critical information can be more detrimental than holding onto a small amount of additional data.

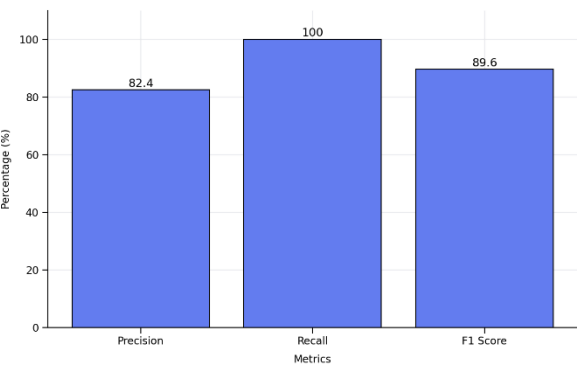


Figure 3 : Evaluation Metrics of Smart Memory Compression Framework.

Taking a closer look at how the scoring factors performed across various tests, it became clear that relevance consistently had the biggest impact on memory retention decisions. Even if information was presented just once, it stuck if it was closely tied to the query or included key contextual clues. Frequency acted as a reinforcing element, especially in single-

topic and audio inputs, where repeated mentions boosted confidence in the importance of a segment. While recency didn't directly dictate decisions, it played a subtle role when similar information popped up multiple times. In those cases, newer versions were preferred, while older ones were either merged or replaced. This strategy helped keep the memory fresh without duplicating content. Across all types of input, the system reliably generated memory representations that were both meaningful and efficient. When compared to storing raw conversation history, the compressed memory was much smaller while still retaining the essential information needed for crafting responses. The differences between

n multi-topic, single-topic, and audio inputs also show that the framework adjusts its behavior based on the conversation's structure rather than sticking to a rigid rule. This adaptability is one of the key strengths observed during testing, allowing the system to effectively navigate a wide range of real-world conversational scenarios.

V. CONCLUSION

This work introduced the Smart Memory Compression Framework, designed to manage memory during those lengthy chats with AI. While I was building and testing this system, one thing became abundantly clear: the real challenge in extended conversations isn't just about having enough memory; it's about dealing with the overload of unnecessary information. Initially, I thought that keeping a larger history would automatically lead to better responses, but it turned out to have the opposite effect. The model slowed down, responses became cluttered with irrelevant details, and it was harder to extract the key information. The framework I developed continuously evaluates the conversation instead of treating it like a static record. Each new interaction is processed, scored, filtered, and then either stored, compressed, or discarded. During testing, this approach consistently yielded better results than simply passing along the raw history. Important details like user preferences, goals, and repeated behaviors were accurately retained, even in longer conversations. Plus, it cut down on redundant phrases and conversational fluff, which directly improved token efficiency. One aspect I monitored closely during testing was whether the system could maintain continuity over time. In many cases, once a piece of information was marked as important, it remained accessible in future interactions without needing to be repeated. This showed that the memory update and retrieval process was working as intended. Another positive outcome was how the

system handled repeated or updated information. Instead of keeping multiple conflicting versions, it merged or replaced them based on relevance and recency, which helped keep the memory structure tidy and organized.

While the system certainly has its advantages, it also comes with a few downsides. In my evaluation, I noticed that it sometimes held onto information that didn't really add value to future conversations. A lot of this was emotional responses or comments that were only relevant at the time, yet they still managed to meet the scoring criteria. This didn't completely throw the system off course, but it did have a slight effect on its accuracy. From a practical perspective, this happened because the scoring function is designed to ensure that no important data gets missed, even if it means keeping some extra bits of information. The project showed that effectively managing conversational memory is more about continuous filtering and updating rather than just hoarding data. The system I created isn't overly complex, but it works well because each component tackles a specific challenge I faced during testing. The end result is a system that uses less memory, reduces unnecessary repetition, and maintains relevant context more consistently over time.

VI. FUTURE ENHANCEMENTS

During our development phase, we discovered that most of our improvements came from really honing in on where the system struggled with those tricky borderline decisions, rather than just focusing on outright failures. One area that definitely needs some attention is how the system handles short-term conversational cues, especially when it comes to expressing emotions. For example, when a user shows excitement or frustration in the moment, the system sometimes flags this as important information and saves it. However, these signals are often fleeting and don't necessarily need to be stored in long-term memory. To improve this, we should work on distinguishing between stable traits and temporary feelings, perhaps by tracking whether a signal appears consistently over time before deciding to keep it. Another limitation I noticed is that once something is saved in memory, it tends to stick around unless it's specifically replaced. This got me thinking about introducing a decay mechanism, where older information gradually loses its relevance if it's not reinforced. In real-life conversations, people's preferences and behaviors change, and our system should reflect that. Right now, while recency helps with updates to some extent, it doesn't actively reduce

the impact of outdated information unless a newer version comes along. Implementing a controlled decay would make the memory more adaptable instead of just a pile of accumulated data. Understanding context is another area that could use some improvement. The current system mainly focuses on what was said, but it doesn't always capture why it was said. During testing, I encountered situations where the same type of sentence could have different meanings depending on the context. For instance, a casual mention versus a more deliberate decision might look similar at first glance but should be treated differently in memory. Adding some level of intent awareness, even through basic heuristics, could really enhance the system's ability to make smarter choices.

Handling audio input definitely brought its own set of challenges. The system did a decent job with transcribed text, but those pesky little transcription errors sometimes messed up the similarity calculations. There were times when key words got misrecognized, which slightly affected how well the information stuck. To boost accuracy, we could either fine-tune the transcription process or make the similarity checks a bit more forgiving when it comes to minor wording differences. One area that really stands out for potential improvement is personalization. Right now, the system is good at storing useful information, but it doesn't quite adapt its behavior based on individual users beyond just basic memory retention.

Over time, it could learn to pick up on patterns in how a specific user communicates, what they often repeat, or what they emphasize. This would allow the memory system to tailor its responses in a way that feels more personal, rather than applying the same rules to everyone. All these enhancements should come from observing the system in action in real-world scenarios, rather than just filling in theoretical gaps. The goal moving forward isn't to complicate things, but to ensure its decisions align more closely with how meaningful conversations naturally evolve over time.

VII. REFERENCES

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin, "Attention Is All You Need," NeurIPS, 2017.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, 2019.
- [3] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," ACL, 2019.
- [4] Nils Reimers, Iryna Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," EMNLP, 2019.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.
- [6] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, Wen-tau Yih, "Dense Passage Retrieval for Open-Domain Question Answering," EMNLP, 2020.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, "Language Models are Few-Shot Learners," NeurIPS, 2020.
- [8] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, Ming-Wei Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," ICML, 2020.
- [9] Stephen Roller, Emily Dinan, Naman Goyal, Da Ju, Mary Williamson, Yinhan Liu, Jing Xu, Myle Ott,

Kurt Shuster, Eric M. Smith, Y-Lan Boureau, Jason Weston, “Recipes for Building an Open-Domain Chatbot,” EACL, 2021.

[10] Chenyan Xiong, Zhuyun Dai, Jamie Callan, Zhiyuan Liu, Russell Power, “Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval,” ICLR, 2021.

[11] James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, Nicholas H. Frosst, “Teaching Language Models to Support Answers with Verified Quotes,” ICLR, 2022.

[12] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, “LLaMA: Open and Efficient Foundation Language Models,” ICLR, 2023.

[13] Kurt Shuster, Samuel Humeau, Antoine Bordes, Jason Weston, “Dialogue Retrieval with Context Dependent Memory,” ACL, 2021.

[14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021.

[15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, Denny Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS, 2022.

[16] Sébastien Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, “Multitask Prompted Training Enables Zero-Shot Task Generalization,” ICLR, 2022.

[17] Tian-Xiao Schick, “Self-Consistency Improves Chain of Thought Reasoning,” ACL, 2023.

[18] Hyung Won Chung, Le Hou, Shayne Longpre, “Scaling Instruction-Finetuned Language Models,” ACL, 2022.

[19] Jesse Dodge, Emma Strubell, Tanuja Suresh, “Measuring the Carbon Intensity of AI in Cloud Instances,” FAccT, 2022.

[20] Yi Tay, Mostafa Dehghani, Dara Bahri, Donald Metzler, “Long Range Arena: A Benchmark for Efficient Transformers,” ICLR, 2021.

[21] Ofir Press, Noah Smith, Mike Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” ICLR, 2022.

[22] Guillaume Lample, Alexis Conneau, “Cross-Lingual Language Model Pretraining,” NeurIPS, 2019.

[23] Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015.

[24] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christopher J. Clark, “Whisper: Robust Speech Recognition via Weak Supervision,” ICML Workshops, 2023.

[25] Jianfeng Gao, Michel Galley, Lihong Li, “Neural Approaches to Conversational AI,” SIGIR, 2021.

[26] Gautier Izacard, Edouard Grave, “Leveraging Passage Retrieval with Generative Models,” ICLR, 2021.

[27] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, Luke Zettlemoyer, “BART: Denoising Sequence-to-Sequence Pre-training,” ACL, 2020.

[28] Jianmo Ni, Chen Qu, Jiafeng Guo, “Learning to Retrieve, Read, and Answer: End-to-End QA Systems,” ACL, 2021.

[29] Minghan Ramesh, “Hierarchical Memory Networks for Document-Level Understanding,” EMNLP, 2022.

[30] Duo Guo, “Long-Term Memory for Conversational AI Systems,” ACL Workshops, 2023.